

Contents

HAL v2.7 — Documentation	1
1. Ce qu'est HAL — en une page	1
2. Architecture	1
3. Modèle de sécurité et RGPD-by-design	2
4. Le routeur IA « 3 vitesses »	3
5. Fonctionnalités (par module)	3
6. Installation et démarrage	4
7. Configuration (variables d'environnement)	4
8. Sauvegardes, clé maître, exploitation	5
9. Lien avec le guide « IA native » et avec Hermès	5
10. Pistes d'évolution (à discuter)	5

HAL v2.7 — Documentation

Assistant pédagogique local-first — filières Froid, Climatisation et Énergies. Application pédagogique développée par un enseignant pour les filières Froid, Climatisation et Énergies — document de présentation technique.

Document de présentation technique — version anonymisée pour diffusion.

Cette documentation décrit l'architecture et le modèle de sécurité de HAL, une application pédagogique *local-first*, à titre d'exemple concret d'IA native respectueuse des données. Elle a été anonymisée : aucune donnée d'élève, aucun secret, aucun nom d'établissement n'y figure. Le guide public « Installer une IA native » reste générique et ne dépend pas de ce fichier.

1. Ce qu'est HAL — en une page

HAL est une **application web locale** qui assiste l'enseignant sur tout le cycle pédagogique : agenda et fiches de séance, distribution et analyse de TP, évaluation par compétences, bilans et suivi par élève — le tout **assisté par une IA exécutée en local**.

Le principe directeur tient en une phrase : **le serveur tourne sur le poste de l'enseignant, le navigateur n'est qu'un écran d'affichage, et aucune donnée d'élève ne quitte la machine**. L'IA fonctionne par défaut sur un modèle local (aucun appel cloud).

C'est l'incarnation concrète de ce que prêche le guide « Installer une IA native » : *local-first* et *RGPD-by-design*, mais poussé bien au-delà d'un simple chatbot — avec chiffrement du nominatif, garde-fou anti-fuite, routeur IA à plusieurs vitesses et verrou cloud.

Repères d'identité. Nom de code HAL (clin d'œil à HAL 9000 — d'où le port **2001**). Version **2.7.0**, dite « la V7 sécurisée » : l'interface V7 (le Mur, calendrier, évaluation) posée sur le backend local-first RGPD-by-design de hal-2.0.

2. Architecture

Couche	Technologie	Rôle
Serveur	Node.js ≥ 20 + Express 5	API locale (port 2001 par défaut), sessions, routage
Données	SQLite (better-sqlite3), 4 bases	inernet (non nominatif), nominatif (chiffré), référentiel, système

Couche	Technologie	Rôle
Chiffrement	AES-256-GCM	champs sensibles (noms, commentaires) chiffrés au repos
Authentification	Argon2id	comptes locaux enseignants, sessions par cookie (12 h glissantes)
IA	Routeur interne → Ollama (local) / Gemini (cloud, optionnel)	analyse de TP, reformulation, synthèses
Interface	HTML / CSS / JS natif (sans framework)	« le Mur » + pages annexes
Extraction documents	mammoth (.docx), pdf-parse (.pdf)	lecture des TP/cours à analyser

Organisation du dépôt

```

src/                serveur, services métier, routes, accès aux bases
  app.js, serveur.js, config.js, journal.js
  routes/           16 routeurs Express (voir §5)
  services/         ~30 services métier (voir §5)
  db/               connexions + migrations (inerweb / nominatif / referentiel / systeme)
public/             interface : le-mur.html (accueil) + admin, bilans, cartes, config,
                   documentation, pedagogie, suivi, tp, visite + modules js/
donnees/           référentiels de diplômes (non nominatifs) : formations.json, progressions.json
scripts/           creer-admin.js, importer-referentiels.js, restaurer.js, hal-
arriere-plan.vbs
tests/             ~40 fichiers de tests (node:test)
data/              [NON versionné] bases, sauvegardes, secrets, logs, emploi du temps
*.bat / *.vbs      lanceurs Windows (voir §6)

```

Important. Le dossier data/ (toutes les données vivantes : bases, sauvegardes, secrets, clé maître) **n'est pas dans l'archive** et n'est jamais versionné. L'archive ne contient que le code, les référentiels non nominatifs et les outils de lancement.

3. Modèle de sécurité et RGPD-by-design

C'est le cœur de HAL. Quatre lignes de défense.

1. Le nominatif ne quitte jamais le poste serveur. Les bases SQLite sont **locales** ; les champs sensibles (nom, prénom, commentaires) sont chiffrés en **AES-256-GCM**. La séparation des bases est volontaire : inerweb (non nominatif) d'un côté, nominatif (chiffré) de l'autre.

2. La clé maître reste sur le serveur. Générée au premier démarrage, **jamais en base, jamais versionnée**. Sans elle, les sauvegardes sont irrécupérables — la copier sur un support sûr est la première consigne d'exploitation.

3. Le navigateur est un simple client d'affichage. Aucune clé, aucun secret, aucune donnée nominative côté front : tout vit sur le serveur.

4. Accès réseau restreint. Écoute par défaut sur 127.0.0.1 (HAL n'est pas exposé au réseau de l'établissement, où le nominatif transiterait en clair). Pour l'usage multi-postes, on passe par **Tailscale** (WireGuard, chiffré de bout en bout) via Securiser-Tailscale.bat ; localhost reste toujours accessible.

Le garde-fou anti-fuite (pseudonymisation). Avant tout envoi vers le cloud, chaque texte passe par verifierSortie() : les noms d'élèves sont remplacés par des codes stables et non

réversibles **ELV-xxxxxxx** (HMAC-SHA-256 dont le sel dérive de la clé maître). La table de ré-identification vit **en RAM uniquement** et ne quitte jamais la tour. La pseudonymisation n'est pas une passerelle vers le cloud : c'est un **filtre anti-fuite**.

Droit à l'effacement. Le miroir froid des sauvegardes a une rétention bornée (30 jours par défaut) pour qu'un élève effacé ne subsiste pas indéfiniment dans les archives.

4. Le routeur IA « 3 vitesses »

Toute requête IA passe par un point d'entrée unique (routeurIA.js), qui choisit une vitesse :

- **V1 — local (défaut, ~0 €)** : modèle **Ollama** local (gemma3:4b par défaut, port 11434). C'est la voie normale, et la seule utilisée dès qu'on suit de vrais élèves.
- **V2 — cloud éco (Gemini)** : gemini-2.5-flash-lite, pour les tâches lourdes **sans donnée nominative**. N'existe que si une clé est posée.
- **V3 — cloud premium** : prévue, non configurée.

Pipeline d'une demande cloud (ordre non négociable) :

0. **Verrou cloud global** — même avec une clé valide, **aucun appel cloud** ne part tant que HAL_CLOUD_AUTORISE n'est pas explicitement à true. (Garde-fou ajouté après audit du 2026-06-12 : évite qu'une clé Gemini oubliée active le cloud par accident. En phase pilote élèves : laisser absent = false.)
1. **Garde-fou RGPD** — alimenté par les listes d'élèves réelles (coffre du Mur + module Suivi) : un texte nominatif ne sort jamais ; ce qui sort est le texte **nettoyé** (codes ELV-xxx).
2. **Budget** — plafond mensuel dur par enseignant (300 centimes = **3 €/prof/mois** par défaut).
3. **Clé** — sans HAL_GEMINI_CLE, la voie cloud n'existe pas.

Tout refus retombe proprement en V1 (local), avec la raison dans le champ repli — **jamais d'erreur** renvoyée à l'utilisateur.

5. Fonctionnalités (par module)

HAL expose 16 routeurs Express et une trentaine de services. Vue fonctionnelle :

Pages principales (public/)

- **Le Mur** (le-mur.html) — page d'accueil après connexion : calendrier, fiches de séance, point d'entrée du quotidien.
- **Suivi** (suivi.html) — suivi par élève, positionnement par compétences (nominatif, chiffré).
- **Bilans** (bilans.html) — synthèses et bilans.
- **TP** (tp.html) — distribution et analyse de TP (cockpit TP, grilles d'évaluation).
- **Cartes** (cartes.html) — cartes / supports.
- **Pédagogie** (pedagogie.html) — outils pédagogiques.
- **Visite** (visite.html) — visite guidée de l'outil.
- **Admin / Config** (admin.html, config.html) — administration, réglages.
- **Documentation** (documentation.html) — aide intégrée.

Services métier (src/services/) — par domaine :

- **Pédagogie / élèves** : eleveService, classesService, suivi (routes), positionnementService, bilanService, assidueService, tpGrilleService, grilleEvalStore, cartesService, pedagogieService, referentielService.
- **IA** : routeurIA, ollamaService, geminiService, gardeFouService, budgetService, assistantService (+ assistantActions, assistantContexte, assistantJournal), ragService, personnaliteService, extractionTexteService.
- **Sécurité / RGPD / exploitation** : cleMaitre, coffreService, authService, auditService, rgpdEleveService, backupService, miroirService, configService, demoService.

Couverture de tests (tests/, node:test) — la suite couvre notamment : RGPD, garde-fou, verrou cloud, anti-hallucination, Ollama, proxy Gemini, suivi élèves, coffre, clé maître, audit, sauvegarde, installation, référentiels. Lancer avec `npm test`.

6. Installation et démarrage

Usage normal (aucune ligne de commande)

1. **Double-cliquer sur Demarrer-HAL.bat.** Le script vérifie Node.js, installe les composants au premier lancement (`npm install`, 1-2 min), démarre HAL en arrière-plan (via `scripts/hal-arriere-plan.vbs`), attend que `/health` réponde, puis ouvre le navigateur sur **`http://localhost:2001`**. Laisser la fenêtre ouverte pendant l'usage.
2. **Première utilisation** : créer le compte administrateur (dans le navigateur). Les référentiels s'importent automatiquement.
3. On arrive sur **le Mur**.

Prérequis : **Node.js** ≥ 20 (depuis `nodejs.org`) et, pour l'IA locale, **Ollama** installé avec le modèle voulu (`ollama pull gemma3:4b`).

Autres lanceurs

- `Arreter-HAL.bat` — arrête le serveur.
- `Installer-Demarrage-Auto.bat` / `Desinstaller-Demarrage-Auto.bat` — (dés)activer le lancement de HAL au démarrage de Windows.
- `Securiser-Tailscale.bat` — configurer l'accès multi-postes chiffré (Tailscale).
- `Ouvrir-HAL.vbs` — ouvrir l'interface sans relancer le serveur.

Mode développement

```
npm install
npm start           # serveur (port 2001 par défaut)
npm test            # suite de tests
npm run creer-admin
npm run restaurer
```

7. Configuration (variables d'environnement)

Tout réglage passe par `src/config.js` (aucun module ne lit `process.env` directement).

Variable	Défaut	Rôle
<code>HAL_PORT</code>	2001	port d'écoute
<code>HAL_HOTE</code>	127.0.0.1	interface d'écoute (mettre l'IP Tailscale pour le multi-postes)
<code>HAL_DATA_DIR</code>	<code>./data</code>	emplacement des données vivantes
<code>HAL_DOCS_RACINES</code>	<code>data/mur/docs</code>	dossiers autorisés pour l'ouverture de documents (séparés par <code>;</code>)
<code>HAL_OLLAMA_URL</code>	<code>http://127.0.0.1:11434</code>	endpoint Ollama
<code>HAL_OLLAMA_MODELE</code>	<code>gemma3:4b</code>	modèle local
<code>HAL_CLOUD AUTORISE</code>	<i>(absent = false)</i>	verrou cloud global — <code>true</code> requis pour tout appel cloud
<code>HAL_GEMINI_CLE</code>	<code>null</code>	clé Gemini (jamais en base, jamais en git)
<code>HAL_GEMINI_MODELE</code>	<code>gemini-2.5-flash-lite</code>	modèle cloud éco

Variable	Défaut	Rôle
HAL_BUDGET_MENSUEL_CENTIMES00		plafond cloud par enseignant (3 €/mois)
HAL_MIROIR_DIR	null	miroir froid des sauvegardes (désactivé si absent)
HAL_MIROIR_RETENTION_JOURS30		rétenion du miroir (droit à l'effacement)
HAL_RAG_URL	—	serveur RAG optionnel

8. Sauvegardes, clé maître, exploitation

- **Sauvegardes** : rotation locale chiffrée (AES-256-GCM), **7 dernières** conservées dans data/sauvegardes/. Cycle nocturne (sauvegarde + miroir) après 2 h du matin.
- **Clé maître** : générée au 1^{er} démarrage, jamais en base, jamais versionnée. **La copier sur un support sûr** — sans elle, les sauvegardes sont irrécupérables.
- **Restauration** : node scripts/restaurer.js <fichier.halbak> (serveur arrêté).
- **Miroir froid** (optionnel) : copie des sauvegardes vers un dossier cible (en prod, un partage réseau privé via Tailscale), avec purge au-delà de la rétention.
- **Accès multi-postes** : exposer le service uniquement sur le réseau privé chiffré (Tailscale) ; localhost reste toujours accessible. Voir docs/DURCISSEMENT.md.

9. Lien avec le guide « IA native » et avec Hermès

- **HAL = la case « local + passif »** de la grille du Chapitre 5 : une IA locale qui *répond* (souveraine, pour les données sensibles), mais en version aboutie (web app RGPD-by-design).
- **Hermès-local** (Chapitre 5, §3.2) peut s'appuyer sur **le même moteur Ollama (port 11434) que HAL** : c'est la case « local + agentique ». Un agent qui *agit* sur les fichiers, branché sur le modèle déjà installé pour HAL — d'où la souveraineté préservée.
- **Hermès (cloud / Claude Code)** reste à part : puissant mais cerveau cloud, donc jamais de données d'élèves.

Point ouvert. Le contenu exact de C:\Hermes\Lancer-Hermes.cmd n'a pas été fourni : la description d'Hermès-local repose sur l'hypothèse « agent → Ollama local ». À confirmer/ajuster si le .cmd fait autre chose.

10. Pistes d'évolution (à discuter)

- **RAG** (ragService, HAL_RAG_URL) : brancher une base documentaire (référentiels, cours) pour des réponses ancrées — utile pour les TP Froid/Clim.
- **Documentation d'exploitation** : formaliser docs/DURCISSEMENT.md (réseau, Tailscale, secrets) et une procédure de reprise après incident (clé maître perdue, base corrompue).
- **Conformité IA Act** : si un module sert un jour à *positionner/orienter* un élève, tracer la supervision humaine (l'enseignant décide) — voir guide public, Ch.1 §2.2.
- **Transférabilité collègues** : un guide d'installation pas-à-pas dédié HAL (distinct du guide public générique) pour qu'un collègue déploie HAL sur son poste.

Documentation générée à partir de l'archive du projet (code, référentiels non nominatifs, lanceurs ; hors data/). Version anonymisée pour diffusion publique.